機器學習實作｜期中考小抄（一）觀念與模型

範圍：第 1–7 週｜單選 25–50 題｜考觀念不考推導＋Python 程式辨識 / 結果預期

1 學習類型與任務

類型	資料	代表任務
監督式	有標籤 y	分類（離散）、迴歸（連續）
非監督式	無標籤	分群、降維
半監督	少量標籤＋大量無標籤	省標註成本
強化式	獎勵訊號	代理人試錯、最大化總獎勵

- 樣本＝一列、特徵＝一欄、標籤 / ground truth＝答案
- **可訓練參數**（$\boldsymbol{w}, b$，從資料學）vs **超參數**（η、epoch、C、k，人先設）
- **特徵選取**挑子集（欄位意義不變）vs **特徵提取**算出新特徵（如 PCA）
- Epoch＝全部訓練資料跑完一輪；**反向傳播**＝用連鎖律從輸出層往回算 loss 對各參數的梯度
- **No Free Lunch**：沒有在所有問題都最好的模型 → 要比較多個演算法
- 流程：先切分 → 前處理只 fit 訓練集 → 訓練 / 選模型（CV）→ 測試集評估一次

2 三個線性分類器的比較

	感知器 PLA	Adaline	邏輯迴歸
淨輸入	$z = \boldsymbol{w}^T\boldsymbol{x} + b$（三者相同）		
激活	階梯函數（不可微）	恆等 $\sigma(z) = z$	sigmoid $\frac{1}{1+e^{-z}}$
損失	依誤分類更新	**MSE**（凸）	**二元交叉熵 BCE**（log loss、負對數概似）
更新時機	只有分錯才更新	依連續誤差更新（分對也可能更新）	同 Adaline
輸出	0 / 1	線性值；0/1 標籤門檻 0.5	機率 $p(y{=}1\|\boldsymbol{x})$

- 決策邊界是**超平面** $\boldsymbol{w}^T\boldsymbol{x} + b = 0$（2 維特徵時是直線）
- PLA（0/1 標籤）：$\Delta w = \eta(y-\hat{y})x$、$\Delta b = \eta(y-\hat{y})$；分對不更新。**Novikoff**：線性可分保證有限步收斂；不可分需設 epoch 上限
- Adaline 從「對或錯」改成「差多少」→ 可微分 → 能用梯度下降
- 邏輯迴歸更新式和 Adaline 形式相同：$\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta\frac{1}{n}\boldsymbol{X}^T(\boldsymbol{y}-\boldsymbol{a})$，差在 a 是 sigmoid 輸出
- 多元分類：**OvR / OvA**（K 個二元分類器；多標籤可獨立判斷，各類機率不必加總為 1）vs **softmax**（類別互斥，機率和＝1）

3 邏輯迴歸細節

- 名字有 regression，做的是**分類**；線性建模的是 logit，不是 p
- 勝算 $\frac{p}{1-p}$（$p{=}0.8 \to 4$）；logit $= \ln\frac{p}{1-p}$ 把 $(0,1)$ 映到全實數；sigmoid 是 logit 的反函數，$\sigma(0) = 0.5$
- sigmoid 輸出是**條件機率** $p(y{=}1|\boldsymbol{x})$＝「已知特徵時屬於正類」的機率，不是預測正確的機率
- BCE：$L = -\frac{1}{n}\sum[y\ln a + (1-y)\ln(1-a)]$；單筆 $y{=}1$ 時 $-\ln a$，**自信地答錯**（$a \to 0$）損失 $\to \infty$
- 損失辨識：Adaline 的 MSE 比較 y 與線性輸出 z；邏輯迴歸的 BCE 比較 y 與機率 $a = \sigma(z)$，**不要把 z 當機率**
- 決策邊界 $z = 0$ 仍是**線性**（sigmoid 不會讓邊界變彎）；要非線性得做特徵工程（如 $|x_1|$、x^2）
- sklearn：`predict_proba` 給機率、`C` 越小正則化越強

4 梯度下降家族

	每次更新用	特性
Full-batch GD	全部樣本	方向穩、每步貴
SGD	1 筆	更新頻繁、路徑吵雜、可**線上學習**（`partial_fit`），每 epoch 要洗牌
Mini-batch	一小批	折衷，可向量化

- 更新：$\boldsymbol{w} \leftarrow \boldsymbol{w} - \eta\nabla L$（沿**負梯度**，梯度指向上升最快）
- 學習率太大 → 跨過谷底、loss 上升**發散**；太小 → 收斂很慢
- 自適應學習率 $\eta_t = \frac{\eta_0}{1+\alpha t}$：前期大步、後期小步
- **特徵縮放**讓等高線接近圓、GD 收斂快很多；標準化≠變成常態分布

5 資料前處理

- 前處理項目：清理、編碼、特徵衍生、縮放、降維（衍生、降維屬特徵工程）
- **黃金法則：先切分、再處理**；轉換器只對訓練集 `fit`，測試集只 `transform`
- **資料洩漏**：測試集資訊（平均、標準差…）滲進訓練 → 分數虛高
- 缺值：`isnull().sum()` 數每欄；刪 `dropna()` 或補（平均、最頻繁值＝類別型、KNN）；空字串 "" 不算缺值
- **順序型**（XL>L>M）→ 手動映射整數；**名義型**（顏色）→ 獨熱編碼，編 0/1/2 會造出假的大小關係
- 獨熱 k 類：含截距的線性模型可丟一欄剩 $k-1$ 欄，避免完全共線（全 k 欄和為 1，與截距相依）；不是所有模型都必須丟欄
- 切分比例：資料少多留訓練、資料多測試比例可小；分類用 `stratify=y`

	正規化 min-max	標準化 z-score
公式	$\frac{x-x_{\min}}{x_{\max}-x_{\min}} \to [0,1]$	$\frac{x-\mu}{\sigma}$ → 平均 0、標準差 1
離群值	**敏感**（其餘被擠扁）	仍受離群值影響；不限制範圍
類別	`MinMaxScaler`	`StandardScaler`（離群多用 `RobustScaler`）

要縮放：靠梯度或距離者（邏輯迴歸、SVM、KNN、SGD、Adaline）；**不必**：決策樹、隨機森林（只比大小）

6 過擬合 / 欠擬合與正則化

	過擬合	欠擬合
別名	**高變異** high variance	**高偏差** high bias
症狀	訓練好、驗證差（有縫）	訓練、驗證都差
對策	加資料、加強正則化、減特徵、降複雜度	加特徵、更複雜模型、減弱正則化

	L_2（權重衰減）	L_1
懲罰	$\frac{\lambda}{2n}\sum w_j^2$	$\frac{\lambda}{n}\sum\|w_j\|$
效果	全部壓小、很少變 0	**稀疏**，很多權重恰為 0 → 可當特徵選取
幾何	約束區域是圓	菱形，角在座標軸上

- λ 越大越趨向欠擬合；sklearn 的 $C \propto 1/\lambda$：C 越小＝正則化越強
- 正則化前通常先標準化（各特徵尺度一致，懲罰才公平）
- **泛化誤差**＝在沒見過的資料上的誤差，用測試集估計；測試集越小估得越不準

7 特徵選取

- 序列特徵選取＝**貪婪搜尋**（局部最佳、次佳解但避開組合爆炸）
- **SFS** 從空集合逐個**加入**；**SBS** 從全集逐個**剔除**使判準 J 損失最小者
- L_1 + `SelectFromModel`；隨機森林的特徵重要度
- 減特徵可降維度詛咒與過擬合、加快訓練

8 常見分類演算法

模型	核心	旋鈕 / 注意
邏輯迴歸	sigmoid 機率、BCE	`C`；權重可解釋
SVM	最大化**間隔 margin**；只靠支援向量決定	`C`；**核技巧**（RBF）處理非線性；`gamma` 太大 → 過擬合
決策樹	是非題切分，選**資訊增益**最大（entropy / Gini 不純度下降）	樹深；不需縮放、可視化
隨機森林	多棵樹（隨機樣本＋隨機特徵）投票	樹數；省事的預設、不需縮放
KNN	**懶惰學習**：訓練只記資料，預測找 k 鄰居投票	通常需縮放；k 小 → 易受雜訊、k 大 → 邊界過平；預測慢

挑選順序：先跑線性基準 → 不夠才上 RBF-SVM / 樹；要解釋選淺樹或邏輯迴歸；資料極大用 SGD 線上學習。

9 看圖判讀（觀念應用）

圖上長相	判讀
loss 對 epoch 越來越大、震盪	可能學習率太大；檢查縮放
loss 下降極慢、像直線	學習率太小，或特徵沒縮放
SGD 的 loss 抖動但整體下降	正常：單筆梯度吵雜
PLA 錯誤數最後歸零	資料線性可分
PLA 錯誤數一直不歸零	可能不可線性分，或輪數不足
訓練 acc 高、驗證 acc 低	過擬合
兩條 acc 都低且貼合	欠擬合
loss 收斂但驗證沒變好	loss 收斂 ≠ 泛化變好

10 易混淆一句話

① 邏輯迴歸損失 = 交叉熵，不是 MSE（MSE 是 Adaline） ② 感知器分對不更新 ③ 不可線性分 → PLA 不收斂 ④ 標準化 ≠ 常態化 ⑤ 對測試集 `fit` = 洩漏 ⑥ 訓練分數本身就低 ≠ 過擬合 ⑦ C 是正則化強度的**倒數** ⑧ L_1 稀疏、L_2 只壓小 ⑨ 樹模型不需縮放、KNN/SVM 通常需縮放 ⑩ 多標籤可用 OvR，softmax 給互斥多類 ⑪ $\boldsymbol{w}$ 定超平面方向（法向量）、b 定離原點遠近 → 只改 b 是平移邊界 ⑫ 激活函數的用途是引入非線性；Adaline 訓練用恆等函數算誤差，分類時仍用閾值 ⑬ 概似是**參數**的函數，不是參數的機率 ⑭ 變異＝換份資料結果差多少（一致性）、偏差＝系統性誤差 ⑮ 多重共線性 → 需要矩陣求逆的方法會病態 ⑯ 獨熱的代價：欄位膨脹、與截距完全共線

11 定義、資料型態與 Iris

- **Mitchell**：任務 T、效能 P、經驗 E —— P 隨 E 提升即「學習」
- **李宏毅**：學習＝找一個函式（參數化模型），一個 x 只能對應一個輸出
- AI（目標）⊃ ML（手段：資料 → 知識）⊃ DL（多層神經網路）
- 學完的用途＝對新資料做**預測**（prediction / inference）

角度	分類	例
結構	結構化	表格、關聯式 DB
	半結構化	JSON、XML、HTML
	非結構化	文字、圖片、影音
數值型	連續	身高、房價
	離散	房間數
類別型	名義（無序）	血型 → one-hot
	順序（有序）	S/M/L → 映射整數

	監督	非監督	強化
回饋	標籤	無	獎勵（常延遲）
目標	預測	找結構	學動作策略

強化式的回饋是獎勵，不是逐筆正確標籤；獎勵可能延遲到整段行動結束。

Iris 資料集

- 150 筆、4 個連續特徵（花萼 / 花瓣長寬，cm）、3 類各 50 筆、無缺值
- 按類別排序（1–50 / 51–100 / 101–150）→ 訓練前要 **shuffle**
- 非監督示範：遮掉標籤後分群；群數相同不代表分群一定正確

工作流程（5 步迴圈）

選特徵＋收集樣本 → 選指標 → 選演算法訓練 → 驗證集評估 → 調超參數 / 換演算法（回到前面）。**測試集一開始就切開、全程不碰**；縮放 / 降維只用訓練集 fit。

12 模型選擇與交叉驗證

- 狹義：**超參數調整**；廣義：**演算法選擇**（用巢狀 CV）
- **Holdout**：訓練 / 驗證 / 測試三份；缺點是結果對切法敏感
- **k 折**：輪流 1 折驗證、$k-1$ 折訓練，取 k 次平均（看平均＋標準差）；常用 $k=10$
- **分層 k 折**：每折保持類別比例 → 分類優先；**LOOCV**：$k=n$（資料很少時）
- **巢狀 CV**：外層評估、內層挑超參數；例 5×2
- 測試集**只在最後用一次**，用來確認，不用來挑選

曲線	橫軸	回答
學習曲線	訓練資料量	多蒐集資料有沒有用
驗證曲線	某超參數值	該選多少（驗證分數峰值）

- 兩線都低且貼合 → 高偏差，**只加資料通常幫助有限**
- 訓練高、驗證低、中間有縫 → 高變異，**加資料可能改善**
- 驗證曲線右端訓練續升、驗證掉頭 → 過擬合區

搜尋	做法	取捨
Grid	窮舉組合	網格內保證找到；次數 $k\times\prod n_j$
Random	從分布抽固定次數	同預算探索更廣
Halving	少資源全評估、逐輪淘汰	節省資源；可能早淘汰好候選

13 混淆矩陣與指標

	預測陽	預測陰
真實陽 P	TP	FN（漏報）
真實陰 N	FP（誤報）	TN

- $\mathrm{ACC}=\frac{TP+TN}{\text{全部}}$，$\mathrm{ERR}=1-\mathrm{ACC}$
- $\mathrm{PRE}=\frac{TP}{TP+FP}$：「說是陽的有幾成真陽」
- $\mathrm{REC}=\mathrm{TPR}=\frac{TP}{TP+FN}$：「真陽抓到幾成」
- $\mathrm{FPR}=\frac{FP}{FP+TN}$；$\mathrm{F1}=\frac{2\,PRE\cdot REC}{PRE+REC}$（調和平均，兩者都高才高）
- **MCC** 用四格，$[-1,1]$，不平衡時比 F1 穩
- **ROC**：掃門檻畫 FPR(x)–TPR(y)；**AUC** 1 完美、0.5 瞎猜；量排序能力
- 固定預測機率，門檻調低 → recall 不減；precision 不保證下降；癌症篩檢重 **recall**，垃圾信過濾重 **precision**
- **微平均**每樣本等權（大類主導）；**宏平均**每類別等權（不平衡適用）
- **類別不平衡**：1% 陽性全猜陰也有 99% ACC → 改指標、過採樣、加權 `class_weight`

例：TP40 FN10 FP20 TN30 → ACC .70、PRE .667、REC .80、F1 .727、FPR .40

14 scikit-learn API

API	方法	例
轉換器	`fit` → `transform`（`fit_transform`）	`StandardScaler`、`SimpleImputer`、`OneHotEncoder`
分類器	`fit(X,y)` → `predict(X)`	`LogisticRegression`、`SVC`、`Perceptron`

```
sc = StandardScaler()
X_tr = sc.fit_transform(X_train)  # ✔ 只 fit 訓練
X_te = sc.transform(X_test)       # ✔ 沿用訓練的 μ、σ
sc.fit_transform(X_test)          # ✘ 重估測試統計量
```

- 學到的參數結尾帶底線：`coef_` 權重、`intercept_` 偏差；建構子給的是超參數
- `transform` 的特徵數須與 `fit` 時相同
- **Pipeline**：訓練時各轉換器 `fit_transform`、最後估計器 `fit`；預測時只 `transform` 再 `predict` → 將原始資料與整條管線交給 CV，逐折 fit
- `make_pipeline(StandardScaler(), LogisticRegression())` 免命名；`Pipeline([('sc',…),('clf',…)])`
- 模組：`model_selection`（`train_test_split`、`cross_val_score`、`GridSearchCV`）、`preprocessing`、`impute`、`compose.ColumnTransformer`、`metrics`
- `cross_val_score(pipe,X,y,cv=10)` 回傳 **10 個分數的陣列**；GridSearch 3×4 組 × cv=5 = 60 次 CV fit（預設重訓再 +1）；`gs.best_estimator_` 取最佳模型，最後只在測試集 score 一次
- `train_test_split(…, test_size=0.3, stratify=y, random_state=1)`

15 Python 基礎（猜輸出）

```
7/2 → 3.5    7//2 → 3    -7//2 → -4   7%3 → 1
2**3 → 8     2^3 → 1 (^ 是 XOR！)
"3"*3 → '333'   [1,2]*2 → [1,2,1,2]   "3"+3 → TypeError
a=[0,1,2,3,4,5]
a[1:4] → [1,2,3]   a[-2:] → [4,5]
a[::2] → [0,2,4]   a[::-1] → [5,4,3,2,1,0]
list(range(1,10,3)) → [1,4,7]   # 不含終點
[x**2 for x in range(5) if x%2==0] → [0,4,16]
len(set("hello")) → 4    bool([]) → False
d={'a':1}; d.get('b',0) → 0; d['b'] → KeyError
t=(1,2); t[0]=5 → TypeError (tuple 不可變)
f"{3.14159:.2f}" → '3.14'    int("3.5") → ValueError
def g(*args, **kw): print(args, kw)
g(1, 2, k=3) → (1, 2) {'k': 3}
def f(x, L=[]): L.append(x); return L
f(1); print(f(2)) → [1, 2] (預設值只建一次)
```

容器	可變	有序	用途
list	✔	✔	增刪暫存
tuple	✘	✔	固定組合；元素皆可雜湊時可當鍵
dict	✔	插入順序	依名字查（雜湊）
set	✔	✘	去重、交集 `&`

- `if/elif` 只執行第一個成立分支（FizzBuzz 要先判 15）；縮排是語法
- `__init__` 建構子；`self`＝物件本身；`self.x=x` 才存成屬性
- `input()` 回傳字串；`random.randint(1,100)` 兩端都含
- `try/except ValueError` 捕捉轉型錯誤；`PermissionError` 是 `OSError` 子類

16 NumPy

```
np.array([1,2,3]).shape → (3,)  # 一維，非行也非列向量
np.array([[1],[2],[3]]).shape → (3,1)
np.array([[[1],[2]]]).shape → (1,2,1)  # ndim=中括號層數
A=np.array([[1,2],[3,4]])
B=np.array([[5,6],[7,8]])
A*B → [[5,12],[21,32]]   # 逐元素
A@B → [[19,22],[43,50]]   # 矩陣乘法 = np.matmul
np.dot([1,2,3],[4,5,6]) → 32
M=np.arange(6).reshape(2,3)  # [[0,1,2],[3,4,5]]
M.sum(axis=0) → [3,5,7]  M.sum(axis=1) → [3,12]
x=np.array([1,5,3,8])
x[x>2] → [5,3,8]  # 布林遮罩
x[[0,3]] → [1,8]    # 花式索引
np.array([1,2.5]).dtype → float64
x[:,np.newaxis].shape → (4,1)
```

廣播：從**最右邊**維度往左比，每對須**相等或其一為 1**；缺的維度補 1；結果取各維最大。

形狀	結果
(4,3)+(3,)	(4,3)
(4,3)+(4,)	ValueError（3≠4）
(4,1)+(3,)	(4,3)
(3,1,2)–(5,2)	(3,5,2)（座標差；還不是距離）
(8,1,6,1)+(7,1,5)	(8,7,6,5)

- `axis=k` 表示**被壓掉**的那一維；`.T` 對一維陣列無效
- ndarray 元素同型別；切片可能不連續；向量化減少 Python 迴圈；矩陣乘法用 `@`

17 Pandas

```
s=pd.Series([10,20,30], index=['a','b','c'])
s.loc['a':'b'].tolist() → [10,20] (含尾端)
s.iloc[0:1].tolist() → [10] (不含尾端)
```

- `DataFrame`＝共用索引的多個 `Series`；`df['col']` → Series；`df[['col']]` → 二維 DataFrame
- `isnull()`（NaN/None/NaT）、`dropna()` 預設刪**列**、`axis=1` 刪欄、`fillna(v)`
- `groupby`＝split–apply–combine；`concat` 沿軸疊、`merge/join` 依鍵配對
- `pd.get_dummies(…, drop_first=True)` → $k-1$ 欄

18 Matplotlib / Seaborn

- 狀態式 `plt.plot/plt.title`（作用在「目前的圖」）vs 物件導向 `fig, ax = plt.subplots()`、`ax.set_title`（多出 `set_`）
- **Figure**＝整張畫布；**Axes**＝一個子圖（不是一條軸）
- `axes[0,2]` 從 0 起算 ＝ `plt.subplot(2,3,3)` 從 1 起算、逐列編號
- `plot` 全點同樣式、快；`scatter` 可逐點設 `s` / `c` / `alpha`、慢
- 成對資料：plot / scatter / bar；分布：`hist`（bins 太寬可能隱藏雙峰）；`pie` 看占比
- `errorbar`、`fill_between` 畫不確定性；`contour/contourf/imshow` 配 `meshgrid`
- 色盤：有序→sequential、有中點→diverging、類別→qualitative；jet 灰階會崩
- Seaborn 架在 Matplotlib 上、直吃 DataFrame：`histplot`、`kdeplot`、`pairplot`、`jointplot`

19 模型內部功能辨識

各片段獨立辨識，對照課堂 0/1 分類；`np` 為 NumPy。

```
z = X @ w + b
# 淨輸入；X:(n,d), w:(d,) → z:(n,)
y_hat = (z >= 0).astype(int)
# PLA 的階梯分類：z=0 也輸出 1
pred = int(xi @ w + b >= 0)
update = eta * (target - pred)
w += update * xi; b += update
# PLA 單筆更新：分對 → update=0
a = z
# Adaline 訓練用線性激活
a = 1 / (1 + np.exp(-z))
# 邏輯迴歸 sigmoid；不是類別標籤
errors = y - a
w += eta * X.T @ errors / len(y)
# 全批次更新；a 的定義決定模型
b += eta * errors.mean()
```

程式 / 線索	辨識重點
`((y-a)**2).mean()`	MSE；Adaline 的 a 是線性輸出
`-(y*np.log(a)+(1-y)*np.log(1-a)).mean()`	BCE；a 是機率（實作需防 log(0)）
外層 epoch、內層逐筆更新	SGD；每輪 n 筆約更新 n 次
每輪對整個 X 算梯度一次	Full-batch；每輪更新 1 次

```
X=np.array([[1.,2.],[3.,4.]])
w=np.array([1.,-1.]); b=0.
z=X@w+b → [-1.,-1.]
(z>=0).astype(int) → [0,0]
eta=0.1; target=1; pred=0
u=eta*(target-pred) → 0.1
w+u*X[0] → [1.1,-0.8]
b+u → 0.1
```

20 API 結果預期與常見陷阱

predict 回傳類別 (n,)；**predict_proba** 回傳 (n,K)，欄位順序看 `classes_`，不是預測正確率。

```
# 假設 clf.classes_ 為 [0,1]
P=np.array([[.8,.2],[.3,.7],[.6,.4]])
P[:,1] → [.2,.7,.4] (類別1機率)
(P[:,1]>=.5).astype(int) → [0,1,0]
(P[:,1]>=.3).astype(int) → [0,1,1]
# StandardScaler：以訓練統計量轉換
sc=StandardScaler().fit([[0.],[2.]])
sc.mean_ → [1.]
sc.transform([[3.]]) → [[2.]]
```

- 100 筆、`test_size=.3` → 訓練 70、測試 30；回傳順序 `X_train,X_test,y_train,y_test`。
- `stratify=y` 保持類別比例；`random_state` 固定切分的隨機性，不保證分數更高。
- `confusion_matrix` 的列＝真實、欄＝預測；labels=[0,1] 時為 `[[TN,FP],[FN,TP]]`，與 §13 排列不同。
- NumPy 的 `*` 是逐元素；Python list 的 `*2` 是重複。兩個 list 做 `*` 或 `@` 都會 TypeError。
- 程式題先看**型別 → shape → 索引 → 運算 → 回傳值**；分清「定義函式」與「呼叫才執行」。